PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Smart Library: Intelligent Digital Library and Book Recommendation System

Team Name: *Team #*	*[Enter team name]* *DSCPP-II-2026-T367*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Goyal, Vanshika – 2510376201 Vanshikagoyal701@gmail.com* *Role: Core Engine (C) + Documentation* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Singhal, Pragati – 2510010644 pragatisinghal05@gmail.com* *Role: Search, Sorting, Main Program, Testing* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Bugana, Prateek – 2510375052 buganaprateek0274@gmail.com* *Role: OOP and Recommendation (C++)* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Not applicable (3-member team)*

PHASE-III PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

The project builds an **Intelligent Digital Library and Book Recommendation System** for colleges and small libraries that still use paper registers or exact-title search. It stores, searches, issues and returns books and suggests unread books from each reader's history. The core engine is written in **C** (linked list, hashing by book ID, binary search tree by title, bubble sort, text files). The user and recommendation layer is written in **C++** (classes, inheritance, polymorphism, Student/Admin menus). The recommender scores unread books by genre (+2) and author (+3). In Phase III a secure **Node.js web portal** was added on the same data files, with dashboard, stock, issue log, due dates and fines. The system is complete and all 17 automated tests pass.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

Layers: Console menu / Web UI → C++ role layer → C core engine → text files (`books.txt`, `users.txt`, `issues.txt`).

C core (library.c): books are nodes of a singly linked list; a hash table (`id % 101`, chaining) finds a book by ID; a BST ordered by title (case-insensitive) gives exact and partial A–Z search; bubble sort on a pointer array sorts by title, author or popularity. Every change is saved to the files immediately.

C++ layer (main.cpp): abstract class `User` with children `Student` and `Admin`; the virtual `menu()` shows the correct role menu. `Recommender` adds +2 per genre match and +3 per author match for unread books and shows the top 5.

Communication: C++ calls C through `library.h` wrapped in `extern "C"`; built with `gcc`/`g++` and a Makefile.

Web portal: Node.js 16+ (no npm packages): HTTP JSON API in `server.js` plus a single-page HTML/JS frontend, using the same data files so both versions stay in sync. Security: salted scrypt password hash, recovery key, lock after 5 wrong attempts, 15-minute idle expiry, one session at a time.

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Core engine in C: linked list of books, load/save `books.txt`, add and delete book, display, issue, return, reading history, issued-books report.	Member 1 (Core Engine + Documentation)
Search and algorithms in C: hash table by ID (`id % 101`), BST title search (exact and partial, A–Z), bubble sort by title, author and popularity.	Member 2 (Search + Main + Testing)
C++ OOP layer: `User`, `Student`, `Admin` classes, virtual `menu()`, login, role-based access, input checks.	Member 3 (OOP + Recommendation)
Recommender: genre +2, author +3, unread books only, top 5, clear message for new users.	Member 3 (OOP + Recommendation)
Makefile, `run_tests.sh` with 17 scripted tests (all pass), sample data in `data_sample/`.	Member 1 (Makefile, data) and Member 2 (tests)
Documentation: project report, viva Q&A, how-to-run, debugging and language/compiler guides.	Member 1 (Core Engine + Documentation)
Web portal (Node.js): secure login, dashboard, books, stock, issue log, students, issue/return desk, due dates, fines, CSV export.	All members (joint work)

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

1. Linking C with C++: C++ renames functions, causing "undefined reference" errors. *Solved* with `extern "C"` in `library.h` and linking with `g++`.

2. Safe input and files: long text, Windows (CRLF) line endings, a missing `books.txt` and the `|` separator could break the program. *Solved* with bounded copies, line-ending clean-up, a start-up message and tests T13–T15.

3. Console and web sharing one data store: console loans have no dates, so the web shows "Not recorded". Web-only data is kept in separate files (`issue_dates.txt`, `book_stock.txt`) that the C program ignores.

4. Performance limits: the BST is unbalanced and bubble sort is $O(n^2)$; fine for hundreds of books. *Plan:* AVL tree and merge/quick sort.

5. Security: console passwords in `users.txt` are plain text (the web owner login is hashed). *Plan:* store hashes for all users.

6. Version control: no repository history in the submitted files. *Plan:* upload to GitHub with milestone commits.

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

1. Working console application in C and C++: login, search by title/ID, sorting, issue/return, history, recommendations, admin tools.
2. Secure web portal (Node.js) on the same data files, with dashboard, stock, issue log, due dates and fines.
3. Data files with sample data (25 books, 3 users) that persist between runs.
4. Automated test suite (17 tests) with saved output.
5. Full documentation: project report, viva Q&A, how-to-run, debugging and language/compiler guides.
6. Makefile and Code::Blocks / VS Code settings for easy build.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

Overall: about 100% of the planned features are implemented and working. The console system builds with 0 warnings (`-Wall -Wextra`) and all 17 automated tests pass.

Ahead of schedule: the web portal with secure login, dashboard, multi-copy stock, due dates, fines and CSV export was not in the original plan (the Phase-I report listed due dates and fines as future scope).

Behind / remaining (minor): adding the repository link and commit history, filling the team details, hashing console passwords, replacing bubble sort and the plain BST with faster structures, and making the web login accept student accounts (currently only the secure owner ID can log in on the web).

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository: [add GitHub link] **Branch:** [main]. *The submitted project archive has no* `.git` *history, so commit IDs must be added from the repository.*

Structure: `src/library.h`, `library.c` (1246 lines, C), `main.cpp` (686 lines, C++); `web/server.js` (403 lines) and `web/public/index.html` (378 lines); `data/`, `data_sample/`, `docs/`, `Makefile`, `run_tests.sh`.

Important milestones to commit: (1) core C engine, (2) hash/BST/sort, (3) C++ classes and recommender, (4) test suite, (5) web portal with secure login.

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
Login and invalid input (T1, T11)	Pass	Wrong login rejected; invalid menu input does not crash.
Search: BST and hash (T2, T3)	Pass	Exact, partial, no-match search; ID 105 found, 999 not.
Sorting (T4a–c)	Pass	Title, author and popularity order verified.
Issue / return, file saving (T5, T12)	Pass	Double issue, wrong ID, double return refused; files updated.
Admin functions (T8, T9)	Pass	Add, duplicate, delete issued/free book, issued report.
Recommendation (T6, T6b, T7)	Pass	Scores 7, 7, 2, 2, 2; read books excluded; no-history message.
Robustness (T13–T15)	Pass	400-character input, CRLF files, missing `books.txt`.
Build check	Pass	0 warnings (`-Wall -Wextra`); docs report 0 valgrind errors.
Web API smoke test	Pass	Login, admin APIs OK; wrong login and no-token call blocked.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

1. Console application (C + C++) – **Completed**.
2. Web portal with secure login – **Completed** (student web login pending).
3. Data files and sample data – **Completed**.
4. Automated test suite, 17/17 passed – **Completed**.
5. Documentation (report, viva Q&A, how-to-run, debugging, language guide) – **Completed**.
6. Makefile and IDE settings – **Completed**.
7. Repository link and commit history – **In progress**.
8. Password hashing for console users, balanced BST and faster sort – **Pending** (future scope).